COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S1
DATE PERFORMED:	JULY 16, 2026	**DATE SUBMITTED:**	JULY 21, 2026
NAME:	AL TWAIN PASION	**INSTRUCTOR:**	ENGR QUEJADO

ACTIVITY NO. 2 **Arrays, Pointers and Dynamic Memory Allocation**

1. PROCEDURE OUTPUT

ILO A: Implement static and dynamic memory allocation & ILO B: Create dynamically allocated objects using pointers and arrays

Table 2-1. Initial Driver Program

INPUT

```cpp
#include <iostream>
#include <string.h>

class Student{
private:
    std::string studentName;
    int studentAge;

public:
//constructor
    Student(std::string newName ="John Doe", int newAge=18){
        studentName = std::move(newName);
    studentAge = newAge;
    std::cout << "Constructor Called." << std::endl;
    };
//deconstructor
    ~Student(){
        std::cout << "Destructor Called." << std::endl;
    }
//Copy Constructor
    Student(const Student &copyStudent){
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }
//Display Attributes
    void printDetails(){
    std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};


int main() {

    Student student1("Roman", 28);
    Student student2(student1);
    Student student3;
    student3 = student2;


    return 0;
}
```

OUTPUT

```
PS C:\Users\TIPQC\Downloads\temp> cd "c:\Users\TIPQC\Downloads\temp\HOA_2\" ; if ($?) { g++ ILO_A.cpp -o ILO_A } ; if ($?) { .\ILO_A }
Constructor Called.
Copy Constructor Called
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
PS C:\Users\TIPQC\Downloads\temp\HOA_2>
```

OBSERVATION

From what I've observed, the code creates a class which is **"Student"** with its private variables being **studentName** and **studentAge** and public members being the constructor of new Student Objects with its default values being *John Doe* and *18*, the deconstructor for when the program ends and it will automatically release the memories being used by the objects, the copy constructor when we want to copy an existing object by allocating memory, and lastly the function to print details of the object, such us their value of **studentName** and **studentAge** .

The sequence of the **int main()** is, firstly it initialized a new object for the **"Student"** class which is **student1** with the **studentName** and **studentAge** values being *Roman* and *28* respectively. After that, it then calls the copy constructor of the class by initializing a new object, **student2**, with the existing object **student1**. This effectively copies the values of **student1** to **student2**. The next line then initialized a new object to the **"Student"** class, which is **student3,** without any set parameters. This then defaulted the values of **studentName** and **studentAge** to *John Doe* and *18* respectively. After that, it then executes a "copy assignment operator" for the values of **student3** from **student2** by assigning **student3** = **student2**, this copies the values from **student2** to **student3**. Now all of our objects have the "same" value of *Roman* and *28*.

When returning to 0, the de-constructor automatically gets called, releasing the memory of every object in the **"Student"** class, and that ends the program. It leaves no memory leaks and is cleaned, which I like. It's a great demonstration of the Rule of Three.

Table 2-2. Modified Driver Program with Student Lists

INPUT

```cpp
#include <iostream>
#include <string.h>

class Student{
private:
    std::string studentName;
    int studentAge;

public:
//constructor
    Student(std::string newName ="John Doe", int newAge=18){
        studentName = std::move(newName);
    studentAge = newAge;
    std::cout << "Constructor Called." << std::endl;
    };
//deconstructor
    ~Student(){
        std::cout << "Destructor Called." << std::endl;
    }
//Copy Constructor
    Student(const Student &copyStudent){
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }
//Display Attributes
    void printDetails(){
    std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};


int main() {

    const size_t j = 5;
    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    return 0;
}
```

OUTPUT

```
PS C:\Users\TIPQC\Downloads\temp\HOA_2> cd "c:\Users\TIPQC\Downloads\temp\HOA_2\" ; if ($?) { g++ ilo_A.cpp -o ilo_A } ; if ($?) { .\ilo_A }
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
PS C:\Users\TIPQC\Downloads\temp\HOA_2>
```

OBSERVATION

Now in the modified code, it utilized a few arrays to add multiple values efficiently. The arrays are the **studentList**, **namesList**, and **ageList**.

The process in the int main begins by initializing the **studentList** as an array object in the **"Student"** class with the size being **j** which is 5. By introducing this, all the values of **studentName** and **studentAge** currently are defaulted to *John Doe* and *18* respectively. The next line, it initialized a string array, **namesList**, that can store up to 5 strings and has the following values; *Carly, Freddy, Sam, Zack,* and *Coddy*. It then initialized an integer array that can also store up to 5 values which are *15, 16, 18, 19, 16*. After that the program returns to 0 and releases the memory used by the **"Student"** class.

I noticed that there was something missing in the code. It actually only initialized the **studentList** with the default values and never did copy the values from the arrays **namesList** and **ageList** into it.

Table 2-3. Final Driver Program

INPUT

```cpp
#include <iostream>
#include <string.h>

class Student{
private:
    std::string studentName;
    int studentAge;

public:
//constructor
    Student(std::string newName ="John Doe", int newAge=18){
        studentName = std::move(newName);
    studentAge = newAge;
    std::cout << "Constructor Called." << std::endl;
    };
//deconstructor
    ~Student(){
        std::cout << "Destructor Called." << std::endl;
    }
//Copy Constructor
    Student(const Student &copyStudent){
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }
//Display Attributes
    void printDetails(){
    std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};

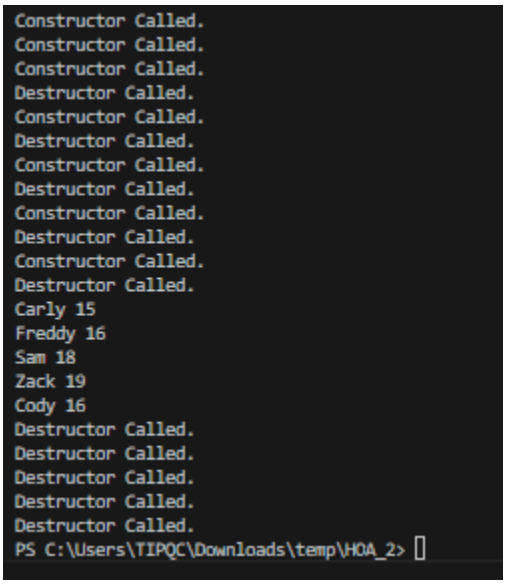
int main() {

    const size_t j = 5;
    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    for(int i = 0; i < j; i++){ //loop A
        Student *ptr = new Student(namesList[i], ageList[i]);
        studentList[i] = *ptr;
        delete ptr;
        ptr = nullptr;
    }

    for(int i = 0; i < j; i++){ //loop B
        studentList[i].printDetails();
    }
    return 0;
}
```

OUTPUT



OBSERVATION

> LOOP A

From my observation, Loop A's purpose is to put the values of **namesList** and **ageList** to the array **studentList** by using the pointer variable **ptr** in a for loop that has a range of **j** in which the value is 5. So in simple terms, it puts these values into the HEAP and moves it back to STACK by the use of "copy assignment operator".

> LOOP B

Just by one look, I can tell that it utilizes the function **printDetails** contained in the **"Student**" class in a for loop to print all of the values in the **studentList** array. This was written after Loop A, so in the output would be what values Loop A has put in the array.

> Modifications

There was only 1 modification I did for the code. Originally in Loop A, the code only initialized the pointer variable **ptr**, BUT, it did not deconstruct this variable after use which leads to a memory leak. So I just put the syntax "delete ptr;" and "ptr = nullptr;" so the leak is patched.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

INPUT

```cpp
#include <iostream>
#include <string>

// to access the private variables, use the getter functions at the very bottom
// to get the subtotal, use the subfSum function
class Fruit {
private:
    std::string fname;
    float fprice = 0;
    int fquantity = 0;

public:

    //constructor
    Fruit (std::string defName = "", float defPrice = 0, int defQuantity = 0) {
        this->fname = defName;
        this->fprice = defPrice;
        this->fquantity = defQuantity;
        std::cout << "constructed!\n";
    }

    //deconstructor
    ~Fruit() {
        std::cout << "deconstructed!\n";
    }

    //copy constructor
    Fruit (const Fruit &copyFruit) {
        this->fname = copyFruit.fname;
        this->fprice = copyFruit.fprice;
        this->fquantity = copyFruit.fquantity;
        std::cout << "copied!";
    }

    //copy assignment operator
    Fruit& operator=(const Fruit &otherFruit) {
        if (this == &otherFruit) {
            return *this;
        }
        this->fname = otherFruit.fname;
        this->fprice = otherFruit.fprice;
        this->fquantity = otherFruit.fquantity;
        std::cout << "rewritten!";
        return *this;
    }

    //to be able to print out the private variables
    std::string getfName() const {
        return fname;
    }
    float getfPrice() const {
        return fprice;
    }
    int getfQuantity() const {
        return fquantity;
    }

    //get the subtotal of the items
    float subfSum() const {
        return fprice * fquantity;
    }
};
```

```cpp
// to access the private variables, use the getter functions at the very bottom
// to get the subtotal, use the subvSum function
class Veggy {
private:
    std::string vname;
    float vprice = 0;
    int vquantity = 0;

public:

    //constructor
    Veggy (std::string defName = "", float defPrice = 0, int defQuantity = 0) {
        this->vname = defName;
        this->vprice = defPrice;
        this->vquantity = defQuantity;
        std::cout << "constructed!\n";
    }

    //deconstructor
    ~Veggy() {
        std::cout << "deconstructed!\n";
    }

    //copy constructor
    Veggy (const Veggy &copyVeggy) {
        this->vname = copyVeggy.vname;
        this->vprice = copyVeggy.vprice;
        this->vquantity = copyVeggy.vquantity;
        std::cout << "copied!\n";
    }

    //copy assignment operator
    Veggy& operator=(const Veggy &otherVeggy) {
        if (this == &otherVeggy) {
            return *this;
        }
        this->vname = otherVeggy.vname;
        this->vprice = otherVeggy.vprice;
        this->vquantity = otherVeggy.vquantity;
        std::cout << "rewritten!\n";
        return *this;
    }

    //to be able to print out the private variables
    std::string getvName() const {
        return vname;
    }
    float getvPrice() const {
        return vprice;
    }
    int getvQuantity() const {
        return vquantity;
    }

    //get the subtotal of the items
    float subvSum() const {
        return vprice * vquantity;
    }
};

```

```cpp
//distinguishing if it's a fruit or a vegetable
class itemType {
public:
    bool isFruit;
    Fruit fItem;
    Veggy vItem;

    itemType (Fruit f) : fItem(f), vItem(), isFruit(true) {}
    itemType (Veggy v) : fItem(), vItem(v), isFruit(false) {}

};

float TotalSum(itemType* list[], int s) {
    float total = 0;
    for (int i = 0 ; i < s ; i++) {
        if (list[i] != nullptr) {
            if (list[i]->isFruit) {
                total += list[i]->fItem.subfSum();
            }
            else {
                total += list[i]->vItem.subvSum();
            }
        }
    }
    return total;
}

int searchIndex(std::string name, itemType* list[], int s) {
    for (int i = 0 ; i < s ; i++) {
        if (list[i] != nullptr) {
            if (list[i]->isFruit) {
                if (list[i]->fItem.getfName() == name) {
                    return i;
                }
            }
            else {
                if (list[i]->vItem.getvName() == name) {
                    return i;
                }
            }
        }
    }
}
```

```cpp
int main() {

    const size_t s = 4;
    itemType* GroceryList[s];

    GroceryList[0] = new itemType(Fruit("Apple", 10, 7));
    GroceryList[1] = new itemType(Fruit("Banana", 10, 8));
    GroceryList[2] = new itemType(Veggy("Broccoli", 60, 12));
    GroceryList[3] = new itemType(Veggy("Lettuce", 50, 10));

    std::cout << "\n\n Grocery list \n";
    for (int i = 0 ; i < s ; i++) {
        if (GroceryList[i] != nullptr) {
            if (GroceryList[i]->isFruit) {
                std::cout << "Fruit: " << GroceryList[i]->fItem.getfName() << ": PHP ";
                std::cout << GroceryList[i]->fItem.getfPrice() << "qty. ";
                std::cout << GroceryList[i]->fItem.getfQuantity() << " = PHP ";
                std::cout << GroceryList[i]->fItem.subfSum() << "\n";
            }
            else {
                std::cout << "Veg: " << GroceryList[i]->vItem.getvName() << ": PHP ";
                std::cout << GroceryList[i]->vItem.getvPrice() << "qty. ";
                std::cout << GroceryList[i]->vItem.getvQuantity() << " = PHP ";
                std::cout << GroceryList[i]->vItem.subvSum() << "\n";
            }
        }

    }

    std::cout << "Total Price: PHP " << TotalSum(GroceryList, s) << "\n";

    //to find lettuce
    std::cout << "\n\n Deleting Lettuce... \n";
    int tempIndex = searchIndex("Lettuce", GroceryList, s);
    delete GroceryList[tempIndex];
    GroceryList[tempIndex] = nullptr;

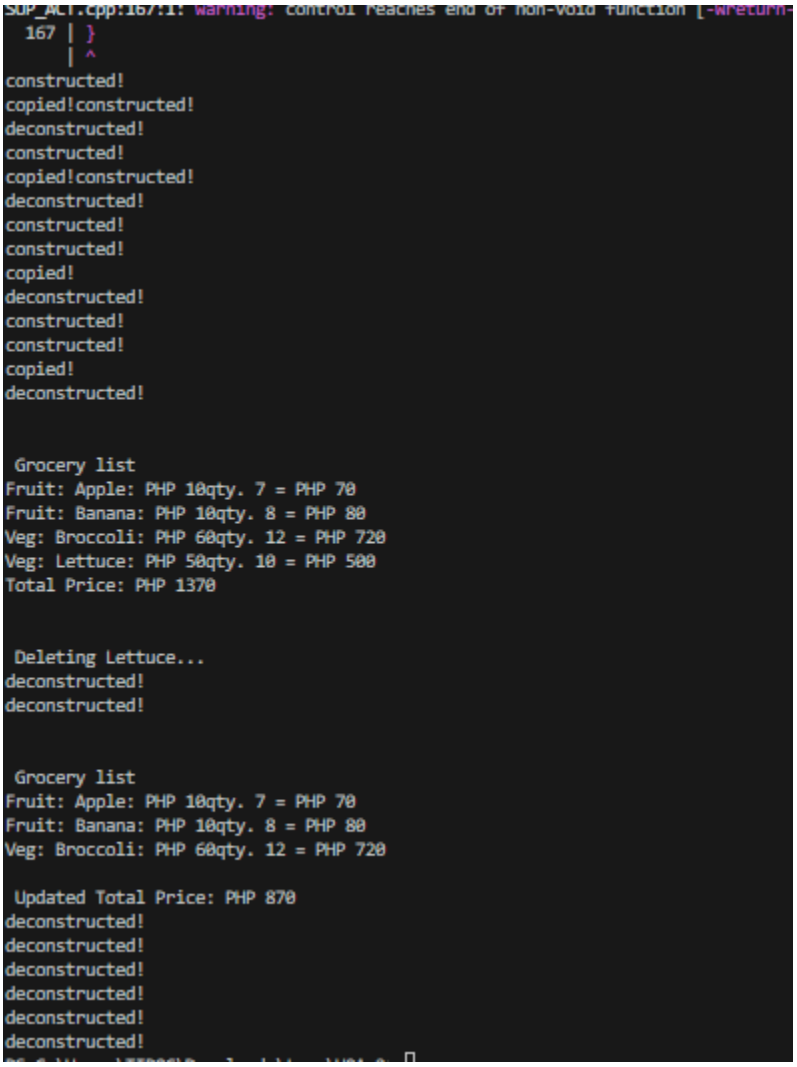
    std::cout << "\n\n Grocery list \n";
    for (int i = 0 ; i < s ; i++) {
        if (GroceryList[i] != nullptr) {
            if (GroceryList[i]->isFruit) {
                std::cout << "Fruit: " << GroceryList[i]->fItem.getfName() << ": PHP ";
                std::cout << GroceryList[i]->fItem.getfPrice() << "qty. ";
                std::cout << GroceryList[i]->fItem.getfQuantity() << " = PHP ";
                std::cout << GroceryList[i]->fItem.subfSum() << "\n";
            }
            else {
                std::cout << "Veg: " << GroceryList[i]->vItem.getvName() << ": PHP ";
                std::cout << GroceryList[i]->vItem.getvPrice() << "qty. ";
                std::cout << GroceryList[i]->vItem.getvQuantity() << " = PHP ";
                std::cout << GroceryList[i]->vItem.subvSum() << "\n";
            }
        }

    }

    std::cout << "\n Updated Total Price: PHP " << TotalSum(GroceryList, s) << "\n";
    for (int i = 0 ; i < s ; i++) {
        if (GroceryList[i] != nullptr) {
            delete GroceryList[i];
            GroceryList[i] = nullptr;
        }
    }

    return 0;
}
```

OUTPUT


SUP_ACT.cpp:167:1: warning: control reaches end of non-void function [-Wreturn-
167 | }
| ^
constructed!
copied!constructed!
deconstructed!
constructed!
copied!constructed!
deconstructed!
constructed!
constructed!
copied!
deconstructed!
constructed!
constructed!
copied!
deconstructed!
Grocery list
Fruit: Apple: PHP 10qty. 7 = PHP 70
Fruit: Banana: PHP 10qty. 8 = PHP 80
Veg: Broccoli: PHP 60qty. 12 = PHP 720
Veg: Lettuce: PHP 50qty. 10 = PHP 500
Total Price: PHP 1370
Deleting Lettuce...
deconstructed!
deconstructed!
Grocery list
Fruit: Apple: PHP 10qty. 7 = PHP 70
Fruit: Banana: PHP 10qty. 8 = PHP 80
Veg: Broccoli: PHP 60qty. 12 = PHP 720
Updated Total Price: PHP 870
deconstructed!
deconstructed!
deconstructed!
deconstructed!
deconstructed!
deconstructed!

OBSERVATION
This activity was exceptionally difficult to apply. In concept, it sounds simple but applying it to code was on a whole 'nother level. This code effectively answered all of the four Problems, but I lacked error handling. The main problem was putting items in different classes based on their category, if they're a fruit or a vegetable. I had to search for a solution because with my current knowledge, I couldn't come up with one. And then I used a member initializer list in a third class to manually initialize items based on their category in the program. This activity's purpose was to learn about dynamic memory allocation, and I achieved that using an array of pointers. In conclusion, I did learn something new from this activity, but I had to search for a solution to solve my own problem.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **PythonTutor AI**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of Syntaxes and their Purpose, Clarification of Understanding**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was only used for me to clarify if my understanding of the code was correct and to explain certain syntaxes that I'm not quite familiar with. Explain the process of the Constructor, Deconstructor, Copy Constructor, and Copy Assignment Operator.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.